How Much Is an Agent's History Worth?

A Law of Evaluation Transfer Across Agent Versions,
Tested in Pre-Registered Studies

Rayen Zargui
Institut Polytechnique de Paris
rayen.zargui@polytechnique.edu

November 2026

Abstract

Every new version of an AI agent is evaluated as if nothing had been learned before, although the outcomes of earlier agents on the same tasks are known. We organise this history as a *graph of agents*—nodes are agent versions carrying their task-level outcomes, edges link versions—and ask how many tasks it is worth. A closed-form law answers: testing m tasks with the graph is as precise as testing $m/(1-\rho^2)$ at random, where ρ^2 is the share of the new version's task-level variance the graph explains (φ^2 with the predecessor, η^2 with the difficulty seen by earlier agents). We turn it into a method: the graph *chooses* the tasks, a cross-fitted estimator *estimates* with calibrated intervals, and the same tasks *decide* keep-or-replace. In eight studies, each pre-registered by a git tag, the law, given the true ρ^2, matched the observed gain within 3.9% (median) over 219 real version transitions from five agent benchmarks; with φ^2 estimated from the 20 tested tasks it still matched on average (5.3%, exploratory), but a single evaluation's estimate erred by 20%. Choosing the tasks gives most of the gain: with the cross-fitted estimator, 20 graph-chosen tasks were worth 32–44 random ones in two confirmations and 90% intervals covered the truth 0.944–0.959 of the time; decisions taken on the same tasks erred 32–54% less often. On 13 agents submitted to AppWorld after its paper, four of them multi-agent systems, the method beat random testing in 24 of 26 evaluations ($\times 1.31$). Agents sharing a scaffold or an agentic pattern fail on the same tasks, though this adds nothing to evaluation efficiency. When results arrive asynchronously and evaluation stops at a deadline, naive estimates overshoot by 6.3 points; a correction learned from older agents did not transfer. Of 43 pre-registered hypotheses, 35 were supported; all failures are reported.

1 Introduction

Agents change constantly. A coding agent ships a new model in March, a new scaffold in May, a new prompt every week. Each change raises the same question—is the new version better than the one in production?—and the standard answer is to re-run the full evaluation suite. On agentic benchmarks this is expensive: a single SWE-bench Verified run executes 500 repository-level tasks, each in its own container [8], and office-automation suites run long multi-step tasks in simulated companies [26].

Yet a team that evaluates a new version is never starting from nothing. It knows how the previous version did on every task, and how every other agent it ever tested did on the same tasks. This paper starts from a simple picture of that knowledge, inspired by the idea of an LLM-maintained wiki of interlinked pages [10]: a *graph of agents* in which each node is an agent version, lineage edges connect successive versions, and each node carries its task-level results. The question we study is quantitative:

When a new node arrives, how many tasks is the rest of the graph worth?

Contributions. This paper makes five contributions.

1. **A law of evaluation transfer** (Section 3). We show that classical results of survey sampling—double sampling [4, 17] and stratification—give a closed-form prediction of the efficiency gain an agent's history provides, $1/(1-\rho^2)$, with two measurable forms of ρ^2: the predecessor's φ^2 and the graph's η^2. The law turns "how much history is transferable" into a single measurable quantity, which can be anticipated from the type of change or estimated from the tested tasks themselves (accurate on average, noisy for a single evaluation; Section 6.1) (Table 2).
2. **A three-move method** (Section 4). The graph chooses the tasks (stratification on the difficulty seen by earlier agents), which provides most of the gain; a cross-fitted generalised regression estimator [21, 28] with effective-sample-size intervals [11] estimates the new version's rate without catastrophic losses; the same tasks decide keep-or-replace and flag whether the decision is sure.
3. **A pre-registered evidence chain** (Section 5). Eight studies—a pilot, six confirmations on data never analysed before, and secondary analyses of new questions—each frozen by a git tag and SHA-256 hashes before the first statistic, with every development step done on previously seen data only and every deviation logged.
4. **Results on 219 real version transitions** (Section 6), including an atlas of what survives a version change, evidence that the graph has structure of its own—an agent's scaffold, more than its model, shapes which tasks it fails—an extension to agentic patterns, multi-agent systems and asynchronous evaluation (Section 6.7), and 8 pre-registered failures reported in full.
5. **A kernel** (Section 7) implementing the graph of agents—an append-only event log, a registry of slots and versions, and the operations `record`, `plan`, `estimate`, `decide`, `lint`—validated by replaying 89 real transitions.

2 Setting

A *task suite* is a fixed set of N tasks with binary outcomes (resolved or not). An *agent version* v has an outcome vector $y_v \in \{0,1\}^N$ and a success rate $p_v = \frac{1}{N}\sum_j y_{vj}$. The *graph* $\mathcal{G}$ at time t contains every version released before t whose outcome vector is fully known, and a lineage edge $v \to v'$ when v' is the next version of the same product (the same agent *slot*). The graph records *evaluation evidence*: it is not an orchestration graph of agents calling one another. A multi-agent system, however it is organised inside, is one node.

When a new version v' arrives, its outcome vector is unknown. We test it on $m \ll N$ *probe* tasks and must (i) estimate $p_{v'}$ with an interval, and (ii) decide whether $p_{v'} \geq p_v$. The baseline is *random testing*: m tasks drawn uniformly, estimated by their success rate (we use a Jeffreys-smoothed mean with Clopper–Pearson-type intervals; with $m = 20$ the smoothing lowers the mean squared error of the plain sample mean by about 9% for rates between 0.3 and 0.7, making the baseline slightly harder to beat, and raises it by about 10% near 5%). We measure every method by its *gain*:

$$\text{Gain} = \frac{\text{MSE(random testing with } m \text{ tasks)}}{\text{MSE(method with } m \text{ tasks)}}, \tag{1}$$

so that m tasks with the method are worth Gain $\times\, m$ random tasks. A gain below 1 is a loss.

3 A law of evaluation transfer

The predecessor. The predecessor v has been measured on all N tasks; the new version on m. This is exactly Neyman's double sampling [17]: a cheap auxiliary variable (here the predecessor's outcome) observed on a large first-phase sample, the expensive variable on a

subsample. Let φ be the correlation between y_v and $y_{v'}$ over tasks (the phi coefficient of their 2×2 flip table).

Proposition 1 (Predecessor law)**.** *Let the predecessor be known on all N tasks and m probes be drawn at random. To first order, the double-sampling regression estimator has variance $S^2(1-\varphi^2)/m + \varphi^2 S^2/N$ when tasks are viewed as draws from a population of tasks [4, ch. 12], where S^2 is the variance of $y_{v'}$. Its gain over random testing is then*

$$\text{Gain}_\varphi = \frac{1}{1 - \varphi^2 + \varphi^2\, m/N}. \tag{2}$$

If instead the target is the success rate on the N tasks themselves (finite population), the second term vanishes and the gain is $1/(1-\varphi^2)$.

The two targets differ by a factor $(1-\varphi^2+\varphi^2 m/N)/(1-\varphi^2)$, close to 1 when $m \ll N$. We pre-registered (2); our replays measure error against the finite-population rate, and the observed gains fall between the two forms (median deviation +1.1% from (2) and -1.1% from $1/(1-\varphi^2)$). The estimator we test is the Bayesian double-sampling estimator of Section 4, which behaves like post-stratification on the predecessor's outcome when the link is informative. Either way, the gain depends on one number, φ^2: how much of the new version's task-level behaviour its predecessor already explains. A minor model update keeps φ^2 high; a rewrite of the agent's orchestration can erase it.

The graph. The rest of the graph carries a different kind of information: which tasks are hard. Sort the N tasks by the difficulty seen by all earlier agents, cut them into m contiguous strata of size $N_h \approx N/m$ (sizes differ by at most one task), and test one task per stratum. Because the strata are built from the history only, the probe outcomes weighted by N_h/N are unbiased for $p_{v'}$; with equal-size strata this is the plain probe mean.

Proposition 2 (Graph law)**.** *Under this stratified design, the gain over random testing is*

$$\text{Gain}_\eta = \frac{(1-m/N)\, S^2/m}{\frac{1}{m^2}\sum_h (1-1/N_h)\, S_h^2} \;\approx\; \frac{1}{1-\eta^2_{\mathcal{G}}}, \tag{3}$$

where S_h^2 is the variance of $y_{v'}$ within stratum h and $\eta^2_{\mathcal{G}}$ is the share of its variance explained by the strata [4, ch. 5].

Both forms are instances of the same statement: *the history is worth $1/(1-\rho^2)$ times the tasks*, where ρ^2 is the fraction of the new version's task-level variance that the graph already explains. The individual results are classical; what is new is the identification of an agent's version history with the first phase of a two-phase design, the use of the graph of all earlier agents as the stratifier, and the empirical test that these closed forms predict the actual gains of agent evaluation (Section 6.1).

4 Method: choose, estimate, decide

Choose. Compute, for each task, the logit of the smoothed success rate of all earlier agents in the graph. Sort, form m strata, draw one task per stratum (Proposition 2). The selection uses the history only, so it is ignorable for model-based inference and keeps the probe mean unbiased.

Graph predictions. We fit a Bayesian logistic regression of the new version's probe outcomes on two graph features—the ecosystem difficulty and the predecessor's outcome—with priors $N(0, 2^2)$ on the intercept and $N(0, 1)$ on the slopes, averaging over the four covariate subsets with weights proportional to their marginal likelihoods (estimated by importance sampling with a multivariate-t proposal). This yields a probability $\hat{\pi}_j$ for every task. The model-based estimate of $p_{v'}$ adds the observed probe outcomes to the predicted outcomes of the $N - m$ untested tasks (finite-population prediction).

Estimate: cross-fitted GREG. Model-based predictions are efficient but can be biased when the model is wrong, which happens for versions that solve almost nothing. The generalised regression (GREG) estimator [21] corrects the predictions with the residuals observed on the probes and is design-unbiased whatever the model. Measuring residuals on the probes used to fit the model makes them too small and the intervals too narrow; we therefore cross-fit [28]: strata are split into two interleaved folds, the model fitted on one fold predicts the residuals of the other, and

$$\hat{p}_{v'} = \frac{1}{N}\sum_{j=1}^{N} \bar{\pi}_j \;+\; \frac{1}{N}\sum_{i \in \text{probes}} N_{h(i)} \left(y_{v'i} - \hat{\pi}_i^{(-k(i))}\right), \tag{4}$$

where $\bar{\pi}$ averages the two fold models and $\hat{\pi}^{(-k)}$ is the model not fitted on fold k. Its variance is estimated by pairing adjacent strata [4, §5A.12], floored at a quarter of the Jeffreys binomial variance, and converted into an interval for a proportion at the effective sample size, with a degrees-of-freedom correction and Jeffreys quantiles [11]; following Brown et al. [3], the lower bound is 0 when no probe succeeds and the upper bound 1 when all do.

Decide. Keep v or switch to v' by the sign of $\hat{p}_{v'} - p_v$, using the model-based estimate on the same graph-chosen probes; the decision is flagged as *sure* when the GREG interval excludes p_v.

Detect regressions sequentially. Probes arrive in batches of 10, up to 80; an alert is raised when the upper 95% bound of the predecessor-based (double-sampling) estimate falls below p_v, and testing stops early when the lower bound exceeds $p_v - 0.08$.

Predecessor-only baseline. To isolate what the graph adds, we also report a Bayesian double-sampling estimator that uses only the predecessor: it averages a model in which the new version ignores the predecessor and a model with separate success rates after predecessor successes and failures, weighted by Beta-binomial marginal likelihoods.

5 Protocol

Pre-registration. Each study was written as a protocol (hypotheses, thresholds, estimators, seeds, data rules) and an analysis script, committed together and frozen by a git tag (`prereg-v1` to `prereg-v6`, `prereg-v8`) and SHA-256 hashes *before any statistic was computed on its data* [18]. A seventh study (`prereg-v7`) is a pre-registered secondary analysis: no new source with enough agents was available, so it re-uses the data of v4–v6 to test a new question (the structure of the graph) on which no statistic had been computed before its freeze. An eighth study (`prereg-v8`, Section 6.7) confirms the method on 13 agents released on AppWorld after the earlier studies, and adds a secondary analysis of agentic patterns on Multi-SWE-bench. Only format audits (file keys, task identifiers, submission names and dates) were allowed before the freeze. Every method change was developed on data already analysed in an earlier study, never on the data of the next confirmation. All deviations—including one accidental display of a single submission's total during a format audit—are logged in the repository. Hashes were re-verified after each run.

Table 1: Pre-registered studies. "Transitions" are real version changes; "Tasks" is the task suite size.

Study	Source	Transitions	Tasks	Nodes are
v1 (pilot)	Sheng & Lu [23]	30	164–5,000	model versions
v2	AppWorld [24]	40	168 / 417	14 agents (4 scaffolds, 4 models)
v3	SWE-bench Verified [8]	89	500	175 public agents
v4	SWE-bench Lite	28	300	84 public agents
v5	Multi-SWE-bench [27] (7 lang.), SWE-bench test and multilingual	33	128–1,587	public agents
v6	LiveCodeBench [7], TheAgentCompany [26]	29	713 / 175	models; office agents
v7 (secondary)	data of v4–v6, re-analysed for graph structure	90	as v4–v6	as v4–v6
v8	AppWorld, 13 later agents; Multi-SWE-bench (patterns)	26 evaluations	168 / 417	4 pattern classes, incl. multi-agent
Total (confirmations v2–v6)		219		

Data. Table 1 lists the sources. Transitions were fixed before any statistic, by rules that differ by source. *Pilot*: five pairs of successive model versions (minor revisions: Llama-3→3.1, Mistral v0.2→v0.3; new generations: Qwen2→2.5, Qwen2.5→3, Gemma 1.1→2) on six benchmarks give the 30 primary transitions; Llama-2→Llama-3 (six more) was pre-specified as secondary and is not counted. *AppWorld*: 40 transitions listed in advance by type of change (model upgrade, scaffold change, change of model family); the graph of a new version contains the other agents except those using its model. *SWE-bench, Multi-SWE-bench and TheAgentCompany*: transitions are defined mechanically from public submission names and dates—within a product lineage, each submission is paired with the latest strictly earlier submission of the same lineage, and the graph of a new version contains only submissions released strictly before it. *LiveCodeBench*: the same rule, with twelve product lines and release dates written down before the freeze (Section 9); models with an uncertain date are excluded from the graphs. Tasks of SWE-bench test that also belong to Verified or Lite (analysed earlier) were removed before the freeze.

Replays. For every transition we simulate the evaluation of the new version 400 times (1,000 in the pilot, 200 in v7 and v8): draw the probes (uniformly, or with the graph's stratified design, independently), apply every estimator to the same probes, and compare with the true rate measured on all N tasks. The primary budget is $m = 20$; $m = 40$ is reported as secondary. Coverage is the fraction of replays whose 90% interval contains the truth.

6 Results

6.1 The law accounts for the gain

Figure 1 plots predicted against observed gains. The predecessor law (2), evaluated at the true φ^2, matches the gain of the double-sampling estimator within 3.9% (median absolute relative error) over the 219 confirmation transitions; the correlation of log-gains is 0.975. The error is 2.7%, 3.6%, 3.9%, 5.8% and 6.4% in the five confirmations, and 2.6% in the pilot: six pre-registered tests of the same law, all within the 20% threshold. The graph law (3), at the true $\eta^2_{\mathcal{G}}$, matches the gain of graph-chosen tasks within 7.7% over 90 transitions (8.8%, 7.7%, 7.2% in v4–v6); the median $\eta^2_{\mathcal{G}}$ is 0.54 on SWE-bench Lite and 0.57 on LiveCodeBench, i.e. the history of other agents explains about half of a new version's task-level variance.

Not only the easy cases. Part of this agreement is easy: for 20% of the transitions the predicted gain is below ×1.1, and predicting "no gain" is then nearly right. On the 75 transitions where the law predicts a gain of at least ×1.5, its median error is 4.3%, against 11.3% for an oracle constant equal to the median observed gain of these same transitions; the graph law errs

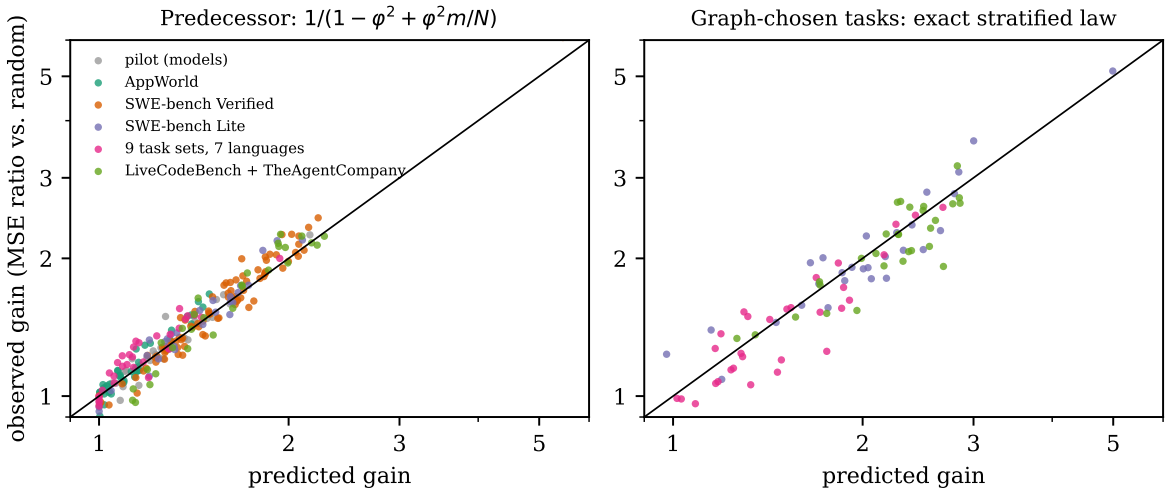


Figure 1: Predicted versus observed gain over random testing, $m = 20$. Left: predecessor law (2), all six studies (249 transitions). Right: graph law (3) for graph-chosen tasks, studies v4–v6 (90 transitions). The diagonal is perfect prediction.

by 7.3% on the 64 transitions where it predicts at least $\times 1.5$. The law therefore tracks how large the gain is, not only whether there is one.

How precise is this test? Each observed gain is itself estimated from 400 replays; a bootstrap over replays puts its relative Monte Carlo error at about 3% (median over transitions). The residual error of the law is therefore of the order of the simulation noise.

With φ^2 estimated from the tested tasks. The tests above use the true ρ^2, measured on all N tasks, which is unknown when a new version is evaluated. In the pilot (pre-registered), plugging the φ^2 estimated from the 20 probes into (2) gave the gain within 4.4% (median), against 2.6% with the true value. In an exploratory, retrospective analysis of v3–v6 whose protocol was committed before it was run (`docs/EXPLORATORY_PHIHAT.md`), we re-drew 20 random tasks 400 times for each of the 179 transitions and computed $\hat{\varphi}^2$ from those tasks alone. Taking the median announced gain per transition, the law erred by 5.3%, against 4.4% with the true φ^2 on the same transitions and 31% for "no transfer"; $\hat{\varphi}^2$ is nearly unbiased (median bias +0.006). A single evaluation is another matter: with 20 tasks $\hat{\varphi}^2$ is noisy (median interquartile range 0.27), so the gain announced by one evaluation erred by 20% (median), and 14% of draws were degenerate (one version all-success or all-failure on the 20 tasks; 55% in v5, where the law erred by 10.0%). The law is thus a reliable guide to the expected value of the history, but forecasting the budget from one small evaluation needs more tasks or a prior from earlier transitions of the same kind (Table 2); a prospective test remains to be done. We did not estimate $\eta^2_{\mathcal{G}}$ from the probes: with one probe per stratum, within-stratum variance is not observed.

6.2 The graph is worth more than the predecessor

Figure 2 summarises the gains. Using only the predecessor gives median gains of $\times 1.12$ (AppWorld), $\times 1.53$ (Verified), $\times 1.47$ (Lite), $\times 1.22$ (v5) and $\times 1.56$ (v6). With random tasks, using the whole graph raises them in four of the five confirmations, to $\times 1.39$, $\times 2.01$, $\times 1.95$ and $\times 1.77$ (on SWE-bench Verified, $p = 7.8 \times 10^{-14}$, paired Wilcoxon); the exception is v5 ($\times 1.12$), where most new versions solve almost nothing (Section 6.4). Letting the graph also choose the tasks

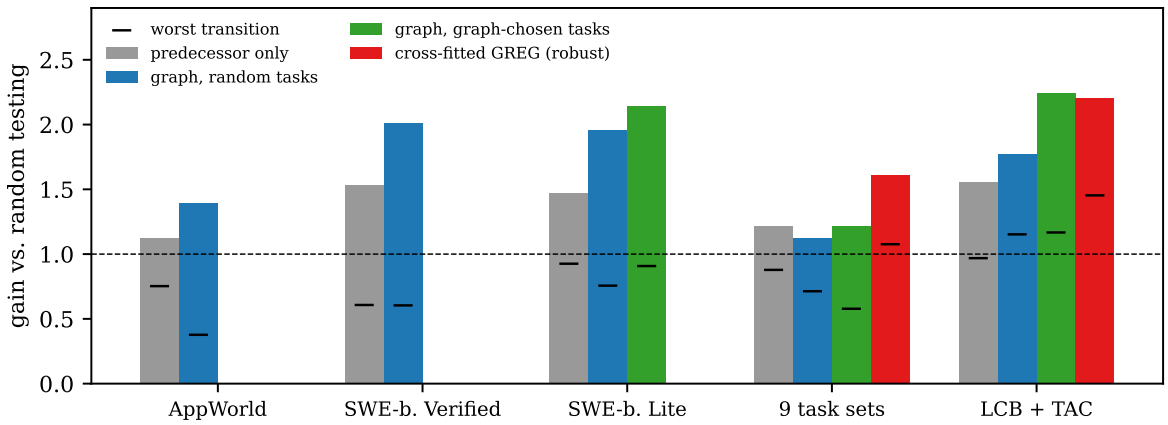


Figure 2: Median gain over random testing ($m = 20$) by study and method; black ticks mark the worst transition. The robust estimator was introduced in v5 (its non-cross-fitted predecessor in v4 is reported in the text).

Table 2: Atlas of transferable information on AppWorld (v2, pre-registered, Kruskal–Wallis $p = 0.0026$).

Type of change	n	median φ^2	gain, predecessor	gain, graph
Model upgrade, same scaffold	8	0.301	×1.47	×1.51
Scaffold change, same model	20	0.081	×1.10	×1.20
Model family change	12	0.045	×1.06	×1.49

gives three gains that must be kept apart. The plain mean of the graph-chosen tasks already reaches ×1.93 on Lite, ×1.27 in v5 and ×2.12 in v6 (Section 6.3). The model-based estimator `graph_v3` reaches ×2.14 on Lite and ×2.25 in v6 (20 tasks worth about 43 and 45 random ones), but it can lose badly on versions that solve almost nothing (Section 6.4). The method's estimator, cross-fitted GREG, reaches ×1.61 in v5 and ×2.20 in v6: 20 graph-chosen tasks are worth 32 and 44 random ones. Study v6 is dominated by LiveCodeBench (27 transitions, ×2.23 with cross-fitted GREG); its 2 TheAgentCompany transitions give ×1.46.

What survives a version change. The AppWorld agents cover 14 of the 16 combinations of four scaffolds and four models, so the type of change can be isolated (Table 2). A model upgrade at fixed scaffold (GPT-4 Turbo → GPT-4o) keeps a median $\varphi^2 = 0.301$; a scaffold change at fixed model keeps 0.081; a change of model family keeps 0.045. *Rewriting an agent's orchestration erases most of the transferable information—its effect is closer to a change of model family than to a model upgrade.* When the predecessor carries nothing, the rest of the graph still does: for family changes the graph gains ×1.49 where the predecessor gains ×1.06.

The lineage edge is real but subsumed. On SWE-bench Verified, a version shares more information with its predecessor than with an unrelated earlier agent of the same success rate (median φ^2 0.367 versus 0.316, pre-registered, $p < 10^{-6}$). But once the difficulty seen by the whole graph is used, adding the predecessor changes the median gain from ×2.006 to ×2.014: statistically detectable ($p = 0.038$) and practically negligible. The value lies in the graph, not in the lineage alone.

6.3 Where the gain comes from (exploratory)

The following comparisons were not pre-registered. Graph-chosen tasks are the analogue of the difficulty-stratified subsets deployed by She and Lin [22]; the plain mean of these tasks is therefore the natural baseline for our estimators. That plain mean already gains $\times 1.93$, $\times 1.27$ and $\times 2.12$ in v4, v5 and v6. The model-based graph estimator improves on it significantly only in v4 ($p = 9.7 \times 10^{-5}$; v6: $p = 0.254$), and GREG only in v5 ($p = 1.4 \times 10^{-5}$; v4, without cross-fitting: $p = 0.895$; v6: $p = 0.268$), where the plain mean's worst case was $\times 0.73$. *Most of the efficiency therefore comes from which tasks the graph selects*—the quantity Proposition 2 predicts—while the estimator mainly buys robustness when versions solve almost nothing, and calibration: the Clopper–Pearson intervals of the plain mean ignore the stratification and over-cover (0.985, 0.984, 0.988). Interval widths tell the same story. On the development data of v5 (Lite, Verified, AppWorld), mean 90% interval widths were 0.31 for random testing, 0.24 for the model-based graph estimator (22% narrower) and 0.31 for the cross-fitted GREG: its intervals are calibrated, but they are not narrower than random testing's, because they stay wide for versions with few successes.

6.4 Robust and calibrated estimation

Model-based estimates can fail on versions that solve fewer than 10% of tasks: on SWE-bench Verified the graph estimator's worst transition was $\times 0.60$ (a submission published with zero resolved tasks), while on the 85 transitions between 10% and 90% its worst case was $\times 1.25$. Study v5 was dominated by this regime (21 of 33 new versions below 10%). There the cross-fitted GREG estimator gained $\times 1.61$ (worst $\times 1.08$; $\times 1.51$ below 10%) with mean coverage 0.950 (minimum 0.812), whereas the model-based estimator's worst case was $\times 0.58$. In v6, where the roles were fixed in advance, the cross-fitted estimator gained $\times 2.20$ (worst $\times 1.45$) with coverage 0.944 (minimum 0.895). The non-cross-fitted GREG of v4 never lost either (worst $\times 1.06$) but its intervals under-covered (minimum 0.625), which motivated cross-fitting.

6.5 Decisions and regression detection

On transitions where the true rates differ by at least 5 points, keep-or-replace decisions from 20 graph-chosen tasks were wrong in 6.5%, 3.9% and 6.0% of replays (v4, v5, v6), against 14.1%, 7.6% and 8.8% with random testing (all pre-registered, $p < 0.05$). In real version histories, the sequential rule detected the 11 SWE-bench Verified versions that regressed by at least 10 points in 92% of replays with 30 probes on average, with 0.8% false alerts on the 73 transitions without a drop.

6.6 The structure of the graph (pre-registered secondary analysis)

So far the graph has been used as a difficulty scale. Does it have structure of its own—do its edges carry information? We fit a one-dimensional model (logit of success = agent ability + task easiness) to all agents of a task set and correlate the standardised residuals of every pair of agents. Multi-SWE-bench offers a crossed design in each of its seven languages: three scaffolds (MSWE-agent, MagentLess, MopenHands) times nine models, 27 agents per language. Development on AppWorld had suggested that the scaffold matters more than the model; the hypotheses were frozen before any structural statistic was computed on these data.

Figure 3 shows the result. Pairs of agents that share a scaffold have more correlated residuals than unrelated pairs in all 7 languages (median difference $+0.060$, $p = 0.008$, Wilcoxon over languages; label permutation significant in 6 of 7), and more than pairs that share only the model (median difference $+0.064$, 7 of 7). Sharing the model also brings agents closer, about half as much ($+0.033$). On LiveCodeBench, models of the same provider are closer than others ($+0.047$, permutation $p = 5.0 \times 10^{-4}$). *An agent's scaffold, more than its model, shapes which*

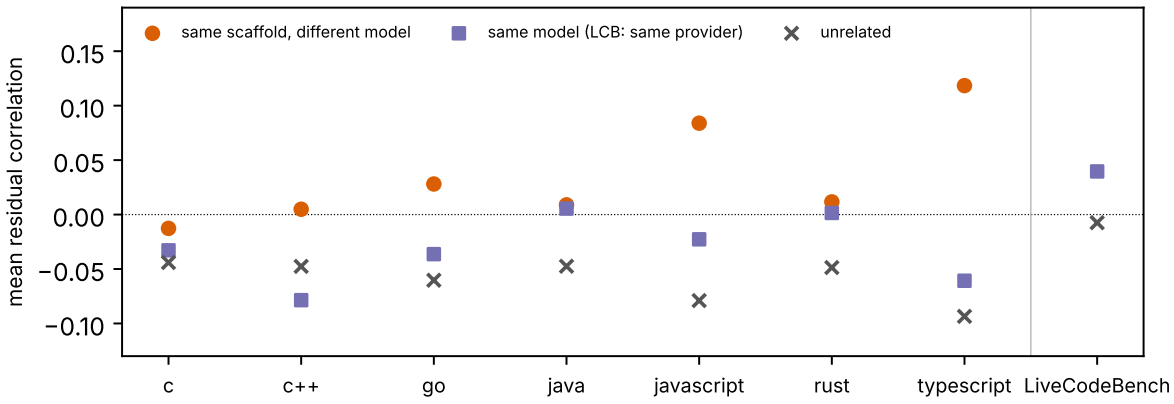


Figure 3: Residual correlation between pairs of agents after removing agent ability and task difficulty, by type of link. Multi-SWE-bench, seven languages, 27 agents each (3 scaffolds × 9 models); LiveCodeBench, one node per model. Only differences between types of pairs are interpreted (the fit induces a common negative offset).

tasks it fails beyond their difficulty—consistent with the atlas of Table 2, where a model upgrade at fixed scaffold keeps most transferable information and a scaffold change erases it.

Does this structure help evaluate a new version? We let each new version attach to the ten earlier agents that behave most like it on its probes (chosen within each cross-fitting fold) and use their behaviour as an extra predictor. Over the 90 transitions of v4–v6, the median gain was ×1.937 against ×1.955 without neighbours (ratio 0.995, $p = 0.654$): not supported. A small gain seen in development (+2% on SWE-bench Verified) did not replicate. The graph's structure is real, but for evaluation its value lies in the difficulty scale it has learned.

6.7 Agentic patterns, multi-agent systems and asynchronous evaluation (v8)

Does the method depend on how an agent is built, and what happens when results do not all arrive? Study v8 uses the 13 agents submitted to the AppWorld leaderboard after its paper (April 2025 to October 2026), never analysed before. Before any outcome was computed, each received a pattern class from its own description: multi-agent systems (LARA,[1] where an explorer writes a plan executed by per-application specialist agents, and IBM CUGA,[2] a hierarchical planner–executor whose controller delegates to specialised sub-agents; two models each), single agents trained by reinforcement learning (four) and prompted single agents with a tool harness (five). Each agent, on each split, is evaluated as a new version whose graph holds every agent released strictly before it (26 evaluations).

Patterns and multi-agent systems. The method beat random testing in 24 of 26 evaluations (median ×1.31, worst ×0.91; $p = 3.7 \times 10^{-7}$), including the multi-agent systems (×1.18 over 8 evaluations, $p = 0.008$), RL-trained agents (×1.22) and prompted single agents (×1.41). The 26 evaluations are 13 agents on two splits; counting each agent once (geometric mean of its two gains, a check added after the run), every agent gained (worst ×1.04, $p = 1.2 \times 10^{-4}$), including all four multi-agent systems, although four units cannot reach significance at 5%. Intervals covered the truth 0.959 of the time (0.949 for multi-agent systems), and the stratified law predicted the gain within 8.1% (median). Gains are smaller than on SWE-bench because AppWorld's graph explains less of a new agent's variance (median $\eta^2 = 0.31$), as the law says they should be. On the 20 evaluations whose rate differs by at least 5 points from the best

[1] https://github.com/Omer212111/LARA
[2] https://arxiv.org/abs/2510.23856

earlier agent, the question "does the new agent beat it?" was answered wrongly in 4.8% of replays by the method and 3.0% by random testing (reported without criterion), another sign that the best decider remains open.

Pattern structure. On Multi-SWE-bench, after removing ability and difficulty, two agents of the same pattern (two agent loops) fail together more than an agent loop and a fixed Agentless-style workflow, with different scaffolds and different models in both cases (median difference $+0.044$, positive in 6 of 7 languages, $p = 0.023$). For the same model, keeping the pattern preserved more transferable information than changing it (φ^2 difference $+0.037$, 5 of 7 languages), but not significantly ($p = 0.109$).

Asynchronous evaluation. Probes usually run in parallel and return in an order set by how many steps the agent takes. Replaying this with each agent's own step counts, stopping when 10 of the 20 probes have returned overestimates the success rate by 6.3 points (median, $p = 3.8 \times 10^{-6}$): slow tasks are mostly hard ones. This is nonresponse bias with outcomes missing not at random [13]: the probes still running are not a random subset of the probes. In development on the paper's 14 agents, a correction learned from the graph—how much more often earlier agents fail where they are slow—removed this bias ($+10.0$ to $+0.1$ points, coverage 0.88). It did not transfer: on the new agents it over-corrected (median bias -8.4 points), erred more than the naive estimate (RMSE ratio 0.86) and covered only 0.770. An exploratory diagnosis explains why: earlier agents failed more often on tasks where they were slow (slowness effect -0.67 on the logit scale), the new agents did not (median $+0.02$), and the mismatch tracks the bias (Spearman -0.83). Correcting for difficulty alone (exploratory, not confirmed) cut the naive error by a factor 1.23 with coverage 0.86. Deadline bias is real; correcting it across agent generations is open. To test the simpler correction without choosing it on data already seen, we froze it in a prospective pre-registration (`prereg-v9`) before any eligible agent existed: it will be run once on the agents released on AppWorld after November 4, 2026, as soon as at least five are available.

Relation to asynchronous agents. The deadline setting is the evaluation-side instance of a pattern now common in agent architectures: an orchestrator launches background workers (asynchronous subagents) and acts on those that have reported, without waiting for the others [20]. Our evaluation harness is such an orchestrator, with one worker per probe. The result suggests a caution for orchestrators in general: when the workers that finish first are those with easier subtasks, aggregating early reports overstates progress. We did not evaluate agents that use asynchronous subagents internally; no public per-task outcomes exist for them yet.

6.8 What failed

Of the 43 pre-registered hypotheses, 8 were not supported (Appendix A). (i) In the pilot, predecessor-based estimates did not reduce decision regret significantly and (ii) sequential detection lacked power at 40 probes; the detection rule was retuned on development data and later confirmed in v2 and v3. (iii) On SWE-bench Verified the graph estimator lost badly on near-zero versions, so the hypothesis of no catastrophic loss failed. (iv) In v5, the model-based estimator with graph-chosen tasks did not stay above random testing on 90% of transitions, because most new versions solved almost nothing. (v) In v7, neighbours learned from the graph's structure did not improve estimation (Section 6.6). (vi–vii) In v8, the graph's correction of deadline bias did not reduce the error and was not calibrated on newer agents, and (viii) the loss of transferable information when the pattern changes was in the predicted direction but not significant (Section 6.7). Two further observations qualify our method. An unbiased estimator is not automatically a good decision rule: in v5, GREG's decisions were wrong in 24.0% of

replays against 13.0% for random testing (9.3% versus 7.6% on differences of at least 5 points). The gap comes from transitions where both versions solve almost nothing: in the extreme case (both below 1%), GREG chose wrongly in 87% of replays and random testing in 0%, because the random baseline's prior pushes such estimates upward and happened to point the right way. And in v6 the pre-registered decider was not the best one: the plain mean of the graph-chosen tasks decided slightly better (4.1% versus 6.0%). The robust gain in decisions comes from *which tasks are tested*; which estimator should decide remains open.

7 The kernel: a graph of agents

The method needs a place where the graph lives. We implement it as a small kernel that observes and does not orchestrate. State is an append-only log of events (slot and version registrations with release dates and lineage, task results with their role and origin, probe plans); five operations act on it: `record` (one result or a whole evaluation), `plan` (the graph chooses m tasks for a new version and stores the plan, which fixes the stratum weights), `estimate` (cross-fitted GREG on the planned probes), `decide` (keep or replace, flagged sure or not), and `lint` (lineage inconsistencies, probes outside the plan, training tasks used as evidence). Each slot gets a Markdown page listing its versions, estimates and intervals. The decision takes its point from the model-based estimator, which decided best in v3, and its interval from cross-fitted GREG; since version 0.2 a decision is flagged sure only when that interval excludes the threshold on the side of the action. Version 0.2 also binds each plan to a hash of the task suite (a plan made on another suite is refused), parses outcomes strictly, keeps every repeated run with its origin and verifier, and requires an even probe budget, since the variance pairs adjacent strata.

Validation. Criteria fixed before the run (on data already analysed, so this tests the software, not the method): replaying the 89 SWE-bench Verified transitions through the kernel alone must reproduce the research gain within 10%, keep coverage above 0.88, and make fewer decision errors than random testing. The kernel (versions 0.1 and 0.2 alike) obtained a median gain of $\times 2.18$, coverage 0.943, and decision errors of 6.0% versus 11.9% for random testing, with no lint finding.

Case study. On the nine real OpenHands versions in SWE-bench Verified, the kernel tested 180 tasks instead of 4,500. Its intervals contained the true rate for 100% of versions and 78% of its decisions were right; both wrong decisions had been flagged as not sure by the kernel itself, and every decision it flagged as sure was right.

8 Related work

Efficient evaluation. Item response theory [19] underlies tinyBenchmarks [14] and anchor points [25], which predict benchmark scores of language models from few items; active testing chooses test points adaptively [12]. For agents, Agent Psychometrics predicts task-level success on coding benchmarks by separating model and scaffold abilities [6]—our residual analysis (Section 6.6) is consistent with a scaffold-specific component; Ndzomga [16] keeps tasks with mid-range historical pass rates; She and Lin [22] study an evolving production agent and deploy difficulty-stratified fixed subsets with IRT-based estimates—the closest design to our task selection, whose gain Proposition 2 predicts; Johnson and Helm [9] estimate new agents' scores from cached traces of reference systems. These works report point accuracy (errors, rank correlations). We add a closed-form law for how much the history is worth (anticipated from the type of change or estimated from the probes), calibrated intervals that survive model failure, the keep-or-replace decision, and pre-registered evidence.

Prediction-powered inference. PPI [1], PPI++ [2], cross-PPI [28] and stratified PPI [5] combine model predictions with a few labels for valid inference; Mani et al. [15] show that the benefit is not free in finite samples. Our estimator is the survey-sampling counterpart (GREG with cross-fitting under a stratified design), where predictions come from the graph of earlier agents rather than from a single predictor.

Version updates. Sheng and Lu [23] show that no single signal predicts which answers regress after an LLM update. We do not predict individual flips; we measure how much the history reduces the cost of estimating and deciding, which is what a team shipping a new version needs.

9 Limitations and threats to validity

Scope. The law assumes probes drawn at random (or within strata) from a fixed task suite; it says nothing about tasks outside the suite. Gains depend on the graph: with fewer than 20 earlier agents the median gain on SWE-bench Verified was markedly lower. Versions that solve almost nothing remain hard for decisions. Several sources share agents or products (SWE-bench splits; 21 of the 33 transitions in v5 are the same model change in different scaffolds and languages), and tasks within a repository are correlated. Release dates in v6 were recorded from memory by the AI assistant (Acknowledgements). LiveCodeBench evaluates models rather than agents, and TheAgentCompany contributes only two transitions.

Statistics. Paired Wilcoxon tests treat transitions as independent units, although transitions of a study share agents and ecosystems; their p-values should be read as descriptive of consistency rather than exact. In v8 each agent contributes one evaluation per split; the per-agent check of Section 6.7 gives the same conclusion. The freezes are recorded by git tags and hashes whose timestamps are self-reported, not certified by a third party; future studies should be registered on a public registry before data access. Each study's hypotheses were tested at level 0.05 without correction across the eight studies; the ledger (Appendix A) reports all 43 tests so that readers can apply their own correction. The law is tested at the true ρ^2; with φ^2 estimated from the probes it was tested in the pilot and, exploratorily and retrospectively, on v3–v6 (Section 6.1); a prospective test, and an estimate of $\eta^2_{\mathcal{G}}$ from the probes, remain to be done.

Baselines. We compare against random testing, a predecessor-only estimator, an IRT-like logistic model on ecosystem difficulty (v3), and the plain mean of graph-chosen tasks (exploratory). We did not re-implement multidimensional IRT or gp-IRT [14, 22]; our claims concern the law, calibration and decisions, not point accuracy against these methods.

Agentic patterns and asynchronous evaluation. A multi-agent system is a single node: we test whether the method evaluates such systems, not how their inner agents interact, and only two multi-agent frameworks (four agents) were available. Pattern classes were assigned from the submissions' own descriptions. The number of environment steps stands in for completion time; asynchronous here means evaluation results that arrive out of order, not the internal architecture of the agents; agents built on asynchronous subagents were not available for evaluation.

Secondary analysis. Study v7 and the pattern analysis of v8 re-use data already analysed for other questions; they are weaker evidence than a confirmation on new data. Residual correlations depend on the one-dimensional model used to remove difficulty.

Kernel. The kernel was validated on SWE-bench Verified, which had already been analysed; this is a test of the software, not new evidence for the method.

10 Conclusion

An agent's history is worth a computable number of tasks. Across 219 real version transitions in five pre-registered confirmations, the law $1/(1-\rho^2)$, given ρ^2, matched that number within a few percent, and a method in which the graph of earlier agents chooses the tasks, estimates with calibrated intervals and decides, divided the number of tasks needed for a given precision by 1.61 to 2.20 (median) in v5 and v6, and by 1.31 on agents released after them (v8). The graph also has structure of its own—scaffolds, more than models, shape which tasks agents fail—but for evaluation its value lies in the difficulty scale it has learned. The method held across agentic patterns, including multi-agent systems; evaluations stopped at a deadline remain biased, and correcting them across agent generations is the next problem, pre-registered prospectively for agents not yet released. Evaluation does not have to start from zero at every release: the graph of agents remembers.

Code and data availability

Code, protocols, frozen analysis scripts with their git tags and hashes, random seeds, raw replay results and the kernel are in the public repository `https://github.com/zarguirayen/cross-version-transfer`. Every number in this paper is generated from the result files by a script (`make_numbers.py`). The benchmark outputs are public: the AppWorld leaderboard bundles, the GitHub repositories `SWE-bench/experiments`, `multi-swe-bench/experiments`, `LiveCodeBench/submissions` and `TheAgentCompany/experiments`, and the data of Sheng and Lu [23].

Acknowledgements

The author conceived the graph-of-agents idea and the research question, designed and directed the pre-registered studies, reviewed the results, and takes full responsibility for the content. Claude (Anthropic), an AI assistant used through Claude Code, wrote the code and ran the experiments under the author's direction, and assisted in drafting the manuscript. In line with arXiv policy, it is not listed as an author.

References

[1] Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. Prediction-powered inference. *Science*, 382(6671):669–674, 2023.

[2] Anastasios N. Angelopoulos, John C. Duchi, and Tijana Zrnic. PPI++: Efficient prediction-powered inference. *arXiv preprint arXiv:2311.01453*, 2023.

[3] Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.

[4] William G. Cochran. *Sampling Techniques*. John Wiley & Sons, 3rd edition, 1977.

[5] Adam Fisch, Joshua Maynez, R. Alex Hofer, Bhuwan Dhingra, Amir Globerson, and William W. Cohen. Stratified prediction-powered inference for hybrid language model evaluation. In *Advances in Neural Information Processing Systems*, 2024.

[6] Chris Ge, Daria Kryvosheieva, Daniel Fried, Uzay Girit, and Kaivalya Hariharan. Agent psychometrics: Task-level performance prediction in agentic coding benchmarks. *arXiv preprint arXiv:2604.00594*, 2026.

[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[9] Ben Johnson and Hayden Helm. Task-efficient agent evaluation using cached traces. *arXiv preprint arXiv:2611.06803*, 2026.

[10] Andrej Karpathy. LLM Wiki (idea file). GitHub gist, 2026.

[11] Edward L. Korn and Barry I. Graubard. Confidence intervals for proportions with small expected number of positive counts estimated from survey data. *Survey Methodology*, 24 (2):193–201, 1998.

[12] Jannik Kossen, Sebastian Farquhar, Yarin Gal, and Tom Rainforth. Active testing: Sample-efficient model evaluation. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.

[13] Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data*. Wiley, 3 edition, 2019.

[14] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[15] Pranav Mani, Peng Xu, Zachary Chase Lipton, and Michael Oberst. No free lunch: Non-asymptotic analysis of prediction-powered inference. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *PMLR*, pages 86296–86335, 2026.

[16] Franck Ndzomga. Efficient benchmarking of AI agents. *arXiv preprint arXiv:2603.23749*, 2026.

[17] Jerzy Neyman. Contribution to the theory of sampling human populations. *Journal of the American Statistical Association*, 33(201):101–116, 1938.

[18] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.

[19] Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, 1960.

[20] Sydney Runkle and Thushanth Bengre. Agents are going async. LangChain blog, https://www.langchain.com/blog/agents-are-going-async, 2026. November 5, 2026.

[21] Carl-Erik Särndal, Bengt Swensson, and Jan Wretman. *Model Assisted Survey Sampling*. Springer, 1992.

[22] Yining She and Lei Lin. Efficient benchmarking in production: A study of an evolving LLM agent. *arXiv preprint arXiv:2609.21267*, 2026.

[23] Jia Sheng and Yiwei Lu. No universal signal predicts sample-level LLM regression under version updates. *arXiv preprint arXiv:2608.13607*, 2026.

[24] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[25] Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics*, 2024.

[26] Frank F. Xu et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

[27] Daoguang Zan et al. Multi-SWE-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*, 2025.

[28] Tijana Zrnic and Emmanuel J. Candès. Cross-prediction-powered inference. *Proceedings of the National Academy of Sciences*, 121(15), 2024.

A Ledger of pre-registered hypotheses

Study	Hypothesis	Criterion (abridged)	Verdict
v1	Variability gate	φ^2 spread ≥ 0.20	supported
v1	H1 law	median error ≤ 0.20	supported (2.6%)
v1	H2 φ^2 depends on pair and benchmark	Holm-corrected tests	supported
v1	H3 lower decision regret	paired test	**not supported**
v1	H4 regression detection	power ≥ 0.80	**not supported**
v2	H1 law (agents)	median error ≤ 0.20	supported (2.7%)
v2	H5 graph > predecessor	Wilcoxon, $\geq$ 90% non-loss, coverage	supported (at the 90% limit)
v2	H2 φ^2 depends on type of change	Kruskal–Wallis	supported
v2	H4 sequential detection	power, probes, false alerts	supported (86%, 52 probes)
v3	H1 law	median error ≤ 0.20	supported (3.6%)
v3	H5 graph > predecessor	as v2	supported
v3	H6 no catastrophic loss	worst gain and coverage $\geq$ 0.80	**not supported**
v3	H7 predecessor adds to graph	Wilcoxon	supported, negligible effect
v3	H2 lineage edge carries information	Wilcoxon vs matched control	supported
v3	H4 detection of real regressions	power, false alerts	supported
v4	H1 law	median error ≤ 0.20	supported (3.9%)
v4	H8 graph law	median error ≤ 0.20	supported (8.8%)
v4	H9 graph-chosen tasks help	Wilcoxon, non-loss, coverage	supported
v4	H10 better decisions	Wilcoxon	supported
v4	H11 GREG no catastrophic loss	worst gain ≥ 0.80	supported
v5	H1 law	median error ≤ 0.20	supported (5.8%)
v5	H8 graph law	median error ≤ 0.20	supported (7.7%)
v5	H9 graph-chosen tasks help	Wilcoxon, non-loss, coverage	**not supported**
v5	H10 better decisions	Wilcoxon	supported
v5	H12 robust and calibrated	worst gain, coverage	supported
v6	U1 estimate: robust and calibrated	worst gain, coverage	supported
v6	U2 estimate: beats predecessor-only	Wilcoxon	supported
v6	U3 decide: beats random testing	Wilcoxon	supported
v6	U4 both forms of the law	median errors ≤ 0.20	supported
v7	S1 structure: shared scaffold	Wilcoxon over 7 languages	supported (7/7)
v7	S2 scaffold more than model	Wilcoxon over 7 languages	supported (7/7)
v7	S3 structure: shared provider	permutation test	supported
v7	S4 learned neighbours help estimation	Wilcoxon	**not supported**
v7	S5 structure's gain is second-order	median ratio ≤ 1.10	supported (trivially, given S4)
v8	N1 method beats random, new agents	median > 1, Wilcoxon	supported ($\times$1.31)
v8	N2 including multi-agent systems	as N1, 8 evaluations	supported ($\times$1.18)
v8	N3 calibrated	coverage ≥ 0.85	supported (0.959)
v8	N4 stratified law	median error ≤ 0.20	supported (8.1%)
v8	A1 deadline bias exists	Wilcoxon	supported (+6.3 points)
v8	A2 graph corrects deadline bias	Wilcoxon, RMSE ratio > 1	**not supported**
v8	A3 calibrated at the deadline	coverage ≥ 0.85	**not supported** (0.770)
v8	P1 same pattern, same failures	Wilcoxon over 7 languages	supported (6/7)
v8	P2 pattern change loses information	Wilcoxon over 7 languages	**not supported** (5/7)

B Development history

Between studies, methods were changed only on data already analysed: the Bayesian model average of the predecessor-only estimator (synthetic worlds, before v1); the graph estimator and the sequential rule (pilot data, before v2); finite-population prediction, model averaging and an identifiability guard (AppWorld and pilot data, before v3); the stratified design and GREG

(SWE-bench Verified and AppWorld, before v4); cross-fitting, the Korn–Graubard interval and the Jeffreys boundary convention (Verified, Lite, AppWorld, before v5). Before v8, the deadline correction (the 14 agents of the AppWorld paper, whose step counts had never been analysed). Variants tried and abandoned (an empirical-Bayes slope prior; fallback rules on the number of successes) are listed in the pre-registration documents.